

Spotify Artist Churn Predictor

Final Report - Introduction to Data Science

Student: Sebastian Vazquez

Course: Introduction to Data Science

Data source: Spotify Web API

Date: 15 June 2026

Project summary

This report documents a complete data science project that collects Spotify top-artist data, engineers user-artist behavioral features, compares four feature selection methods, trains a Random Forest model, and deploys the result through a Dockerized FastAPI prediction endpoint.

Table of Contents

1. Executive Summary
 2. Assignment Requirements and Project Compliance
 3. Problem Definition
 4. Data Source and Collection Workflow
 5. Dataset Construction
 6. Churn Label Definition
 7. Feature Engineering
 8. Exploratory Data Analysis
 9. Feature Selection
 10. Modeling and Evaluation
 11. API Deployment and Dockerization
 12. Main Findings
 13. Business Interpretation
 14. Ethical Considerations
 15. Limitations
 16. Conclusion
- Appendix A. Reproducibility Guide
- Appendix B. API Examples

1. Executive Summary

This project builds a Dockerized machine learning application that predicts artist listening disengagement risk using data collected from the Spotify Web API. The application does not attempt to predict Spotify subscription cancellation. Instead, it addresses a more measurable churn problem: whether a user is likely to stop listening to a specific artist who was historically relevant to them.

The project collected data from 5 valid authorized Spotify users. For each user, the app retrieved top artists across three listening windows: long term, medium term, and short term. The final dataset contains 328 user-artist records. Each row represents one relationship between a user and an artist.

The target label, `churned_artist`, was engineered because Spotify does not provide a direct artist churn label. An artist was labeled as churned if the artist appeared in a user's long-term or medium-term top artists but did not appear in the user's short-term top artists. This represents a practical proxy for recent artist disengagement.

Metric	Value
Valid users	5
Final rows	328
Churned class proportion	55.79%
Retained class proportion	44.21%
Model	Random Forest Classifier
Test F1-score	0.7609
Cross-validation F1-score	0.7193

Table 1. Executive summary of the final project results.

2. Assignment Requirements and Project Compliance

The final project required the development of a customer or user churn predictor application using data fetched from a public API or public web source. The project also required feature generation, comparison of four feature selection methods, a Dockerized API, and a short report explaining which features matter most and why.

Requirement	How this project satisfies it
Public data source	Spotify Web API with OAuth authorization and user-top-read scope.
Feature generation	Ranking, presence, normalized score, historical affinity, and rank-change features were engineered from raw API lists.
Minimum feature pool	12 model candidate features were generated before feature selection.
Filter methods	Variance threshold, ANOVA F-test, and correlation analysis were applied.

Wrapper method	Recursive Feature Elimination with Logistic Regression was applied.
Decision Tree importance	A single Decision Tree was trained and used to rank features.
Random Forest importance	A Random Forest model was trained and used to rank features.
Prediction API	FastAPI exposes /health, /features, and /predict endpoints.
Dockerization	Dockerfile and docker-compose.yml allow the app to run with docker compose up --build.
Report	This report explains the methodology, features, model, results, interpretation, ethics, and limitations.

Table 2. Mapping of assignment requirements to implemented components.

3. Problem Definition

3.1 Why Artist Churn Instead of Subscription Churn?

Spotify does not expose a simple public column such as `subscription_churned` or `cancelled_account` through the Web API. It also does not provide raw play counts through the top artists endpoint. For this reason, predicting subscription cancellation directly would not be technically realistic with the available data. The project therefore defines a narrower and measurable churn angle: artist listening disengagement risk.

This type of churn is still meaningful in a music streaming setting. A user may not abandon Spotify, but they may stop listening to artists, genres, or musical styles that previously defined their preferences. Detecting this change can help a platform improve recommendations, surface relevant new releases, or react to early signs of content disengagement.

3.2 Unit of Analysis

The unit of analysis is a user-artist relationship. This means that each row in the modeling dataset represents one anonymized Spotify user and one artist that was historically relevant to that user. The model predicts whether that specific artist is at risk of being disengaged for that specific user.

one row = one anonymized user + one artist

4. Data Source and Collection Workflow

4.1 Spotify Web API

The project uses the Spotify Web API endpoint for a user's top artists. For each authorized user, the application collects top artists from three different time ranges: `long_term`, `medium_term`, and `short_term`. Each request asks for up to 50 artists.

```
/me/top/artists?time_range=long_term&limit=50
/me/top/artists?time_range=medium_term&limit=50
/me/top/artists?time_range=short_term&limit=50
```

4.2 OAuth Authorization

The app uses Spotify OAuth authorization. Each participant authorizes the application, and the app receives an access token that allows it to call the Spotify top artists endpoint on behalf of that user. The required scope is user-top-read. Since the app is used in development mode, users must be explicitly registered in the Spotify Developer Dashboard allowlist before their data can be collected.

4.3 Raw Data Storage

For every successfully authorized user, the /collect endpoint saves one raw JSON file inside data/raw. The user identifier is anonymized before storage. These raw JSON files are treated as the source of truth for rebuilding the processed dataset.

```
data/raw/top_artists_<anonymized_user_id>.json
```

5. Dataset Construction

The dataset is built by reading all valid raw JSON files from data/raw. For each user, the script combines the long-term, medium-term, and short-term artist lists into a single user-artist table. Artists that appear only in the short-term window are excluded from the training base because they represent new interests rather than historically relevant artists that could be churned.

Dataset property	Final value
Valid authorized users	5
Final user-artist records	328
Number of generated columns	20
Candidate model features before selection	12
Final selected model features	7
Target variable	churned_artist

Table 3. Final dataset construction summary.

The final dataset size is appropriate for this assignment. It is large enough to produce a meaningful binary classification exercise while still being small enough to inspect and explain in detail.

6. Churn Label Definition

Spotify does not directly label artist churn. Therefore, the target variable had to be engineered from observed user behavior. This reflects a common real-world data science problem: the business definition of churn must be converted into a measurable rule.

Final churn rule

churned_artist = 1 if the artist appears in long_term or medium_term top artists, but does not appear in short_term top artists.
Otherwise, churned_artist = 0.

This target should be interpreted carefully. It does not prove that the user permanently abandoned the artist. It means the artist disappeared from recent top listening behavior after being historically relevant. This is a proxy for disengagement, not a confirmed behavioral fact.

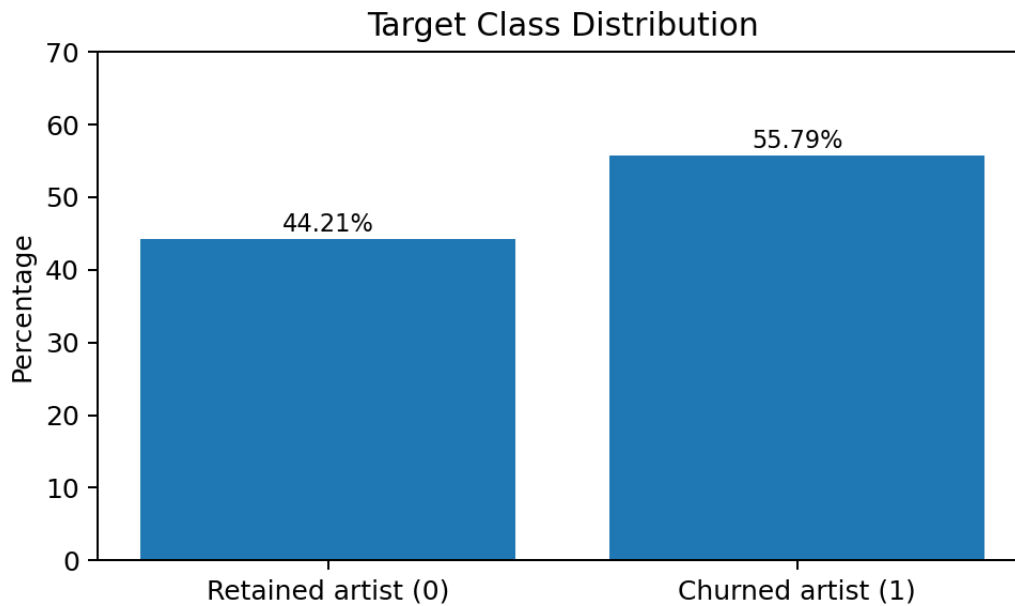


Figure 1. Distribution of the engineered churn label.

Class	Meaning	Proportion
0	Artist retained in recent listening	44.21%
1	Artist not present in recent listening	55.79%

Table 4. Target class balance.

7. Feature Engineering

The central data science task in this project is feature engineering. The raw Spotify API response contains lists of artists by time range. Those raw lists are not directly suitable for modeling. They must be transformed into numerical and binary indicators that represent user-artist behavior.

The feature engineering strategy is based on three questions: Was the artist historically relevant? Did the artist remain relevant in the medium-term window? Did the strength of the relationship decline over time?

Feature type	Feature	Meaning
Ranking	long_rank	Position of the artist in long-term listening.
Ranking	medium_rank	Position of the artist in medium-term listening.
Ranking	short_rank	Position of the artist in short-term listening; kept for audit, not used for training.
Binary	appears_long_term	Whether the artist appears in long-term top artists.
Binary	appears_medium_term	Whether the artist appears in medium-term top artists.
Binary	appears_short_term	Whether the artist appears in short-term top artists; used to define target, not used for training.
Change	rank_change_long_to_medium	Change in artist rank between long-term and

		medium-term windows.
Aggregation	historical_best_rank	Best rank achieved across long-term and medium-term history.
Aggregation	historical_avg_rank	Average historical rank across long-term and medium-term windows.
Aggregation	historical_presence_count	Number of historical windows where the artist appears.
Binary	is_long_only	Whether the artist appears only in long-term history.
Binary	is_medium_only	Whether the artist appears only in medium-term history.
Binary	is_present_in_both_historical_windows	Whether the artist appears in both long-term and medium-term windows.
Score	long_rank_score	Normalized long-term rank score.
Score	medium_rank_score	Normalized medium-term rank score.
Score	historical_affinity_score	Average historical score summarizing affinity strength.

Table 5. Engineered features.

Data leakage prevention

short_rank and appears_short_term were excluded from the model inputs because they directly define the target label. Including them would allow the model to learn the answer instead of learning behavioral patterns.

8. Exploratory Data Analysis

The exploratory analysis focused on class balance, feature distributions, and differences between retained and churned artists. The target balance is suitable for classification. The most useful behavioral differences appeared in ranking-related and historical-affinity-related features.

The class distribution is close enough to balanced that the model can learn both classes. The churned class is slightly more common, but not dominant. This reduces the risk of a model that simply predicts the majority class.

The analysis also showed that ranking features and normalized score features have enough variation to be useful. Some binary features were less useful on their own but still valuable in combination with rank-based indicators.

9. Feature Selection

After generating candidate features, the project applied four feature selection approaches. The goal was not simply to keep every engineered feature, but to identify which features were consistently useful across different selection strategies.

9.1 Filter Methods

Filter methods were used as a fast initial evaluation. Variance Threshold identified whether features had enough variation to be informative. ANOVA F-test measured the relationship between each feature and the churn label. Correlation analysis helped detect redundant variables.

9.2 Wrapper Method: RFE

Recursive Feature Elimination was applied using Logistic Regression. This method repeatedly fits a model and removes weaker predictors until a smaller feature set remains. RFE is useful because it evaluates feature usefulness in the context of a model rather than only as isolated variables.

9.3 Decision Tree Importance

A Decision Tree classifier was trained to compute feature importance. The advantage of a single tree is interpretability. However, a single tree can be unstable, so its ranking was compared against Random Forest results.

9.4 Random Forest Importance

Random Forest importance was used as the most stable tree-based ranking because it averages many decision trees. Features that ranked highly in Random Forest and also appeared in other methods were considered stronger candidates.

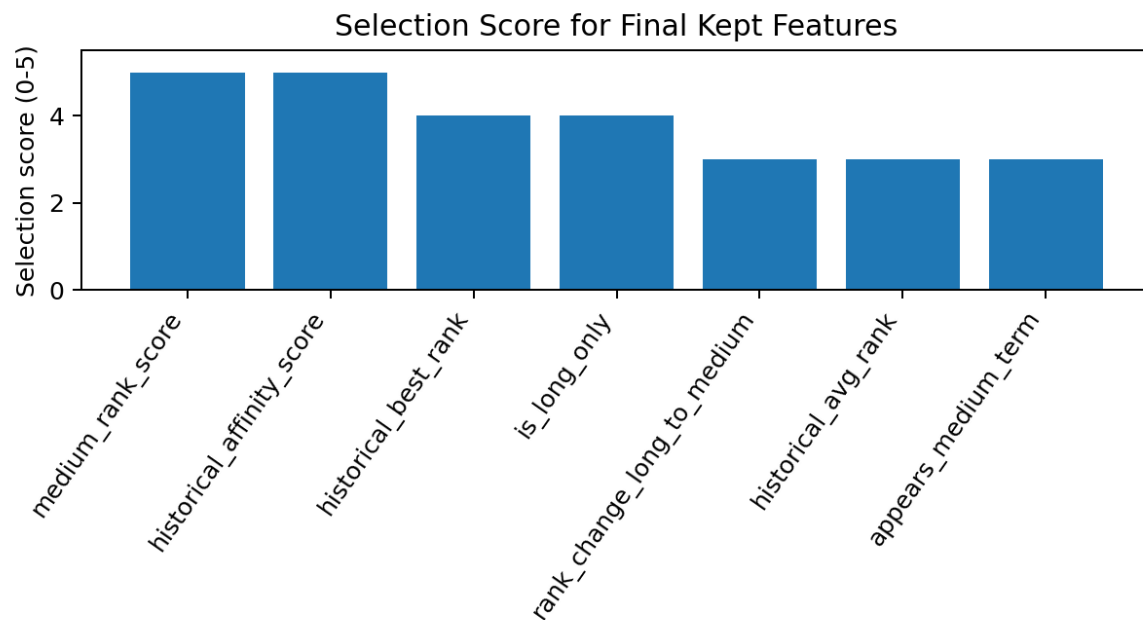


Figure 2. Selection score for features kept in the final model.

Feature	ANOVA	RFE	DT	RF	Score	Decision
medium_rank_score	1	Yes	1	1	5	Keep
historical_affinity_score	4	Yes	3	5	5	Keep
historical_best_rank	6	Yes	2	4	4	Keep
is_long_only	2	Yes	4	6	4	Keep
rank_change_long_to_medium	7	No	5	2	3	Keep
historical_avg_rank	5	No	7	3	3	Keep
appears_medium_term	3	Yes	8	7	3	Keep
long_rank_score	10	No	6	8	1	Drop
historical_presence_count	9	No	8	9	1	Drop

appears_long_term	12	No	8	10	1	Drop
is_present_in_both_historical_windows	8	No	8	11	1	Drop
is_medium_only	11	No	8	12	1	Drop

Table 6. Feature selection comparison across four methods.

10. Modeling and Evaluation

10.1 Final Model

The final model is a Random Forest Classifier. This model was selected because it handles non-linear relationships, works well with numerical and binary features, provides feature importance values, and is more stable than a single Decision Tree.

10.2 Final Selected Features

- medium_rank_score
- historical_affinity_score
- historical_best_rank
- is_long_only
- rank_change_long_to_medium
- historical_avg_rank
- appears_medium_term

10.3 Performance Metrics

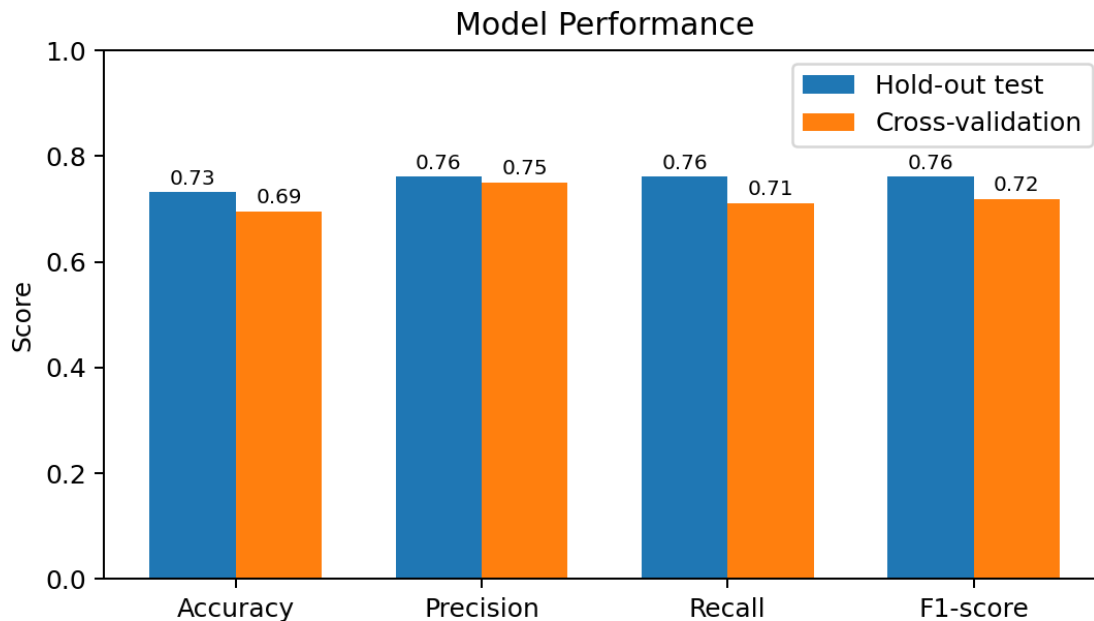


Figure 3. Hold-out test and cross-validation performance metrics.

Metric	Hold-out test	Cross-validation
Accuracy	0.7317	0.6950

Precision	0.7609	0.7497
Recall	0.7609	0.7114
F1-score	0.7609	0.7193

Table 7. Final model performance metrics.

The F1-score is the most important metric for this project because it balances precision and recall. Accuracy alone could be misleading if one class were much more common than the other. The final cross-validation F1-score of 0.7193 indicates that the model has reasonable predictive performance for an introductory data science project with a small behavioral dataset.

11. API Deployment and Dockerization

The trained model was deployed through a FastAPI application. The model is loaded from `app/model.pkl` when the API starts. The expected feature list is stored in `app/model_features.json` and returned by the `/features` endpoint.

Endpoint	Purpose
<code>/health</code>	Checks whether the API is running and the model is loaded.
<code>/features</code>	Returns the list of features expected by the model.
<code>/predict</code>	Accepts feature values and returns a churn prediction with probability.
<code>/login</code>	Starts Spotify OAuth authorization.
<code>/callback</code>	Receives the authorization code and exchanges it for a token.
<code>/collect</code>	Collects top artists from the authorized user and saves raw JSON.

Table 8. API endpoints.

11.1 Dockerization

The project includes a `Dockerfile` and `docker-compose.yml` file. The Dockerized app can be started with the following command:

```
docker compose up --build
```

The Dockerized API was tested successfully using the `/health`, `/features`, and `/predict` endpoints. This confirms that the application can run in a reproducible environment rather than depending only on the local Python setup.

12. Main Findings: Which Features Matter Most and Why?

medium_rank_score

This was the strongest feature. It captures how strongly the artist appears in medium-term listening. If an artist remains strong in the medium-term window, the user is less likely to have disengaged from that artist.

historical_affinity_score

This feature summarizes the overall historical strength of the user-artist relationship. Strong historical affinity helps distinguish core artists from occasional listening.

historical_best_rank

This feature captures the best historical position an artist achieved. An artist that was once highly ranked but is no longer recent provides a clear disengagement signal.

is_long_only

This binary flag identifies artists that only appear in long-term history. Such artists often represent older preferences that did not continue into more recent windows.

rank_change_long_to_medium

This feature detects movement in user interest. A decline between long-term and medium-term rank can indicate early disengagement before the artist disappears from short-term listening.

historical_avg_rank

This provides a stable historical rank measure that reduces the impact of a single unusually high or low rank.

appears_medium_term

This indicates whether the artist remained relevant after the long-term window. Medium-term presence generally reduces churn risk.

Overall finding

Artist churn risk is mainly associated with weakening or disappearing listening affinity across time windows. The model is not learning global artist popularity; it is learning the change in relevance between a specific user and a specific artist.

13. Business Interpretation

A streaming platform could use a model like this to support personalized retention and recommendation strategies. The model identifies artists that were historically meaningful to a user but may no longer be part of recent listening behavior.

- Recommend new releases from artists with high churn probability.
- Suggest similar artists when a user seems to be moving away from a previous preference.
- Create personalized playlists that reconnect users with historically important artists.
- Surface concerts, collaborations, or albums related to artists the user used to listen to.
- Analyze whether disengagement is happening at the artist level, genre level, or broader content category level.

The practical value is not only predicting churn, but also creating actionable interventions that improve user experience while respecting user autonomy.

14. Ethical Considerations

The model predicts probability, not certainty. A user classified as at risk of losing interest in an artist should not be treated as a confirmed churn case. This distinction matters because recommendation systems can influence user behavior.

Ethical issue	Explanation
Privacy	User identifiers are anonymized and raw account identifiers should not be exposed.
Transparency	Users should understand that recommendations may be personalized based on listening behavior.
Misclassification	A user may still like an artist even if that artist does not appear in recent top artists.
Over-personalization	Too much personalization can narrow content diversity and reduce discovery.
Manipulation risk	The model should support recommendations, not pressure users into specific behavior.

Table 9. Ethical considerations.

15. Limitations

1. The dataset contains only 5 authorized users, even though it contains 328 user-artist records.
2. The target label is engineered and not directly provided by Spotify.
3. Spotify top artists are based on calculated affinity, not raw play counts.
4. The project predicts artist disengagement, not subscription cancellation.
5. The dataset may not generalize to the broader Spotify user population.
6. Some Spotify artist fields, such as follower count or popularity, were not reliable in the collected responses and were excluded.
7. The user-artist rows are not fully independent because multiple rows can come from the same user.

Despite these limitations, the project demonstrates the complete workflow expected in an introductory data science final project: API-based data collection, feature generation, feature selection, model training, API deployment, and Dockerization.

16. Conclusion

This project successfully developed a Dockerized FastAPI application that predicts artist listening disengagement risk using Spotify user behavior data. The final dataset contained 328 user-artist records from 5 authorized users. The target label was engineered by comparing historical and recent listening behavior.

The final model selected 7 features and achieved a test F1-score of 0.7609 and a cross-validation F1-score of 0.7193. The strongest predictors were related to medium-term rank, historical affinity, best historical rank, long-term-only presence, and rank change over time.

The central conclusion is that artist disengagement can be modeled as a decline in user-artist affinity across listening windows. This makes the project a useful demonstration of how behavioral signals can be transformed into a churn prediction problem.

Appendix A. Reproducibility Guide

A.1 Project Structure

```
spotify-churn-predictor/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── spotify_auth.py
│   ├── scraper.py
│   ├── features.py
│   ├── model.py
│   ├── model.pkl
│   └── model_features.json
├── notebooks/
│   └── eda_and_selection.ipynb
├── data/
│   ├── raw/
│   └── processed/
│       ├── spotify_artist_churn_dataset.csv
│       ├── feature_selection_comparison.csv
│       └── model_metrics.json
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
├── .env.example
├── README.md
└── FINAL_REPORT.docx
```

A.2 Rebuild Dataset

```
python app/features.py
```

A.3 Train Model

```
python app/model.py
```

A.4 Run Dockerized API

```
docker compose up --build
```

Appendix B. API Examples

B.1 Health Check

```
Invoke-RestMethod -Uri "http://127.0.0.1:8000/health"
```

B.2 Get Model Features

```
Invoke-RestMethod -Uri "http://127.0.0.1:8000/features" | ConvertTo-Json -Depth 5
```

B.3 Prediction Request

```
$body = @{
    medium_rank_score = 0.80
    historical_affinity_score = 0.85
    historical_best_rank = 5
    is_long_only = 0
    rank_change_long_to_medium = 10
}
```

```
    historical_avg_rank = 8  
    appears_medium_term = 1  
} | ConvertTo-Json
```

```
Invoke-RestMethod -Uri "http://127.0.0.1:8000/predict" -Method POST -Body $body -ContentType  
"application/json"
```